Methodology

1. System Design and Processing Flow

The backend uses a deterministic, stage-wise analysis pipeline implemented in Flask. For each candidate resume, the pipeline executes:

1. Resume parsing and identity extraction.
2. GitHub profile and repository data acquisition.
3. Repository-level technology detection.
4. Resume-vs-GitHub skill matching.
5. Repository quality and code-health scoring.
6. Contribution behavior analysis.
7. Final multi-factor scoring.
8. Report synthesis and artifact generation.

The same pipeline is used for both single-file and bulk processing; bulk mode applies the pipeline independently per resume and aggregates outputs.

2. Data Sources and Inputs

2.1 Resume Input

Accepted input formats are PDF and DOCX.

1. PDF text is extracted primarily with pdfplumber, with fallback to PyPDF2.
2. PDF GitHub profile extraction is performed with PyMuPDF link and text URL scanning.
3. DOCX text is extracted using python-docx.

The parser extracts:

1. Candidate name (heuristic from early non-contact lines).
2. Email and phone via regex.
3. GitHub username via URL pattern matching with exclusion filtering.
4. Resume-declared skills using keyword matching and skill normalization.

2.2 GitHub Input

Given a GitHub username, the backend retrieves:

1. User profile metadata.
2. Public repositories (paginated up to practical cap used by service).
3. Language distribution estimates.
4. Contribution summary signals.
5. Repository contents, file contents, and commits as needed by downstream analyzers.

3. Repository Sampling Strategy

Different analyzers use ranked subsets of repositories to optimize signal quality and runtime.

1. Tech stack analysis: non-fork repositories sorted by stars plus recency, up to top 50.
2. Repository quality analysis: non-fork repositories sorted by stars plus recency, up to top 10.
3. Contribution analysis: owned repositories prioritized by stars/recency (top subset), forked repositories analyzed separately as external contributions.

This design emphasizes representative, active repositories while controlling API and compute cost.

4. Technology Detection Method

Technology inference is repository-centric and evidence-based.

4.1 Evidence Channels

1. File extensions (for language/tool inference).
2. Config/build files (for framework/runtime inference).
3. package.json dependency graph inspection.
4. requirements.txt package mapping.
5. Pattern signatures in source and config files.

4.2 Confidence and Aggregation

For each detected technology in each repository, the analyzer stores:

1. Confidence score.
2. Evidence snippets.
3. Technology type label.

Cross-repository aggregation computes usage frequency as:

$$ \text{usageFrequency}(t)=\frac{\text{repos containing }t}{\text{repos analyzed}}\times 100 $$

5. Resume-GitHub Skill Matching Method

Skill matching operates on normalized canonical forms with exact and fuzzy fallback.

5.1 Matching Procedure

1. Normalize resume skills and detected GitHub skills.
2. Attempt exact canonical match.
3. Attempt fuzzy containment/word-overlap match.
4. If unmatched, classify as missing skill.
5. Detect extra GitHub skills not present in resume under strict confidence/usage/evidence gating.

5.2 Authenticity Score

Let M be matched resume skills, U be unmatched resume skills, and E be extra discovered skills.

Base match ratio:

$$ r=\frac{|M|}{|M|+|U|} $$

Base score:

$$ S_{base}=100r $$

Extra-skill bonus:

$$ B=\min(2|E|,10) $$

Authenticity score:

$$ S_{auth}=\min(100,\max(0,S_{base}+B)) $$

6. Repository Quality and Code Health Method

Repository quality is computed per repository, then calibrated and portfolio-aggregated.

6.1 Active Category Weights

Current production weights:

1. Documentation: 0.15
2. Code organization: 0.30
3. Commit quality: 0.25
4. Code health: 0.30 (optional if analyzable)

If an optional category is unavailable, weights are re-normalized over active categories.

6.2 Documentation Subscore

Documentation combines README quality and structural documentation signals.

README quality is computed from:

1. Word-count tiers.
2. Presence of installation/usage/features/contributing sections.
3. Badge and code-block presence.
4. Optional readability bonus (textstat).

Category score composition includes README-derived score and auxiliary documentation indicators.

6.3 Code Organization Subscore

A structure label (excellent/good/basic/minimal) is derived from directory patterns and converted to a base score. A bounded richness bonus is added from directory count.

6.4 Commit Quality Subscore

Commit quality combines temporal frequency and message quality:

$$ S_{commit}=0.45,S_{freq}+0.55,S_{msg} $$

Frequency uses activity volume, recency concentration, and consistency (gap variance). Message quality rewards good length, conventions, issue references, and action verbs; it penalizes noise and repetition.

6.5 Code Health Subscore

Code health is best-effort, language-aware static analysis over sampled source files with depth and file-count bounds.

1. Python: Radon MI+CC if available; AST fallback otherwise.
2. JavaScript/TypeScript: esprima-based complexity estimate.
3. Java: javalang-based complexity estimate.

For polyglot repositories, language-specific results are file-count weighted with capped per-language weights.

6.6 Per-Repository Weighted Score

For active categories i with score s_i and weight w_i:

$$ S_{repo}=\frac{\sum_i s_i w_i}{\sum_i w_i} $$

6.7 Portfolio Calibration and Grade Adaptation

Raw per-repo quality scores are calibrated to reduce overly strict static behavior.

For portfolio mean μ and standard deviation σ (floored for stability):

$$ z=50+\frac{(x-\mu)}{\sigma}\cdot 15 $$

$$ S_{cal}=\text{clamp}(0.75x+0.25z+3,0,100) $$

Dynamic grade thresholds are derived from calibrated distribution and forced monotonic.

6.8 Portfolio-Level Repository Quality

Each repository receives an importance weight from stars and recency:

$$ w_r=\min(1+0.08\cdot \text{stars}_r+\text{recencyBonus}_r,5) $$

Portfolio repository-quality score:

$$ S_{rq}=\frac{\sum_r w_r S_{cal,r}}{\sum_r w_r} $$

7. Contribution Analysis Method

Contribution analysis separates owned repository development from fork-based external contributions.

7.1 Owned Repository Contributions

For prioritized owned repositories, the analyzer computes:

1. User commit count.
2. Total commit count.
3. Ownership percentage.
4. Commit message quality label.
5. Active/inactive status by recency.

7.2 External Contributions

Forked repositories are scanned for user-authored commits to estimate open-source contribution behavior.

7.3 Temporal Activity Features

Monthly and weekday activity distributions are built from commit timestamps, including recent-commit totals and streak-related consistency indicators.

8. Final Multi-Factor Scoring Method

Final score is a weighted linear combination of five normalized components:

1. Skill authenticity.
2. Repository quality.
3. Commit activity.
4. Tech stack strength.
5. Profile signal.

Weights are:

$$ w_{auth}=0.60,;w_{rq}=0.10,;w_{ca}=0.10,;w_{ts}=0.10,;w_{ps}=0.10 $$

Overall score:

$$ S_{overall}=w_{auth}S_{auth}+w_{rq}S_{rq}+w_{ca}S_{ca}+w_{ts}S_{ts}+w_{ps}S_{ps} $$

8.1 Component Derivations

1. Skill authenticity: max(authenticity score, match percentage) for display consistency.
2. Repository quality: calibrated portfolio quality score.
3. Commit activity: recency-dominant blend of recent and total commits.
4. Tech stack: breadth (capped skill count) and confidence blend.
5. Profile signal: completeness, depth, active volume, consistency, and account maturity blend.

8.2 Rating Bands

The numeric overall score is mapped into ordinal recommendation bands from Below Average to Excellent using fixed thresholds.

9. Report Construction and Artifact Generation

A structured report object is synthesized with:

1. Candidate summary.
2. GitHub profile summary.
3. Technical analysis.
4. Skill matching details.
5. Repository insights.
6. Code quality report.
7. Contribution report.
8. Scoring analysis and recommendation.
9. Detailed metrics.

In addition, the pipeline generates a local deep-scoring HTML artifact after each successful analysis run. This artifact includes stored and derived values, calibration traces, and per-repository component-level computations.

10. Error Handling and Robustness

The backend uses guarded execution and fallback paths at each stage.

1. Missing GitHub URL triggers a domain-specific exception and user-facing message.
2. GitHub API HTTP errors are mapped to explicit diagnostic responses.
3. Optional analyzer dependencies degrade gracefully when unavailable.
4. Upload and temporary extraction files are cleaned after processing.
5. Bulk mode isolates per-file failures to avoid cascading pipeline abortion.

11. Experimental Reproducibility Notes

1. Results depend on live GitHub state at fetch time.
2. Token presence affects API rate-limit resilience.
3. Optional packages (textstat, radon, esprima, javalang, rapidfuzz) influence analyzer fidelity.
4. Repository sampling caps and depth constraints are fixed in code and should be reported with experiments.

12. Scope Constraints of Current Backend Method

Current repository-quality methodology intentionally excludes test/CI/license-driven scoring in active category computation and centers on documentation, organization, commit quality, and optional code health. This is a deliberate modeling choice in the present backend version and should be disclosed in the paper as a design constraint.